

USER MANUAL

Body Movement Detection

1. Introduction

This project monitors visible body movement through a webcam using MediaPipe pose estimation. It identifies body landmarks, compares their positions across frames, and classifies individual body regions as moving, stationary, or uncertain.

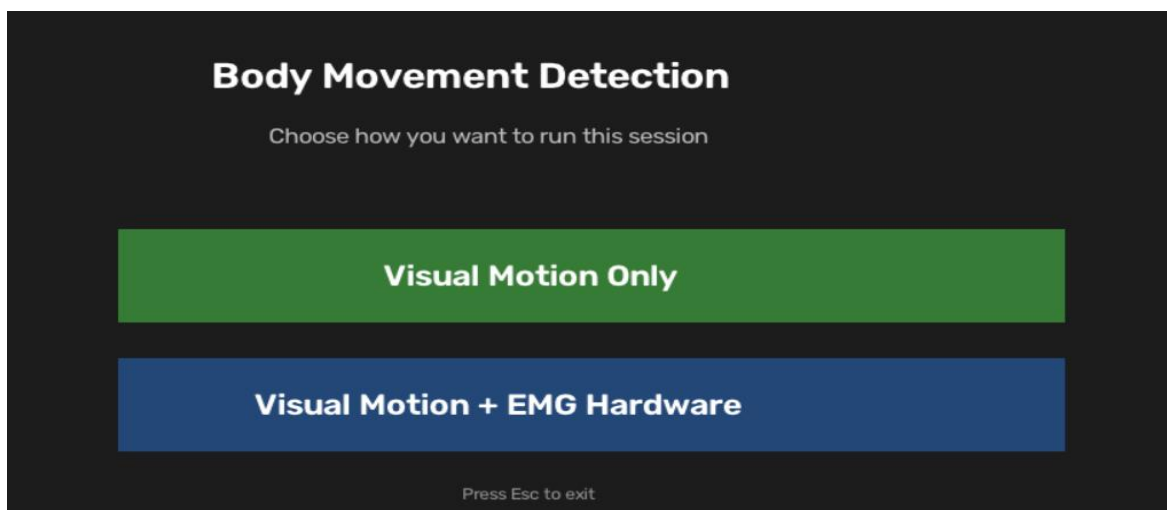
The main challenge is distinguishing actual movement from small changes caused by landmark estimation noise. The implementation addresses this through position smoothing, body-size normalization, region-specific thresholds, and repeated evidence before changing movement states. It also considers partial visibility so that an unavailable measurement is not automatically interpreted as stillness.

The project monitors the head, left and right eye regions, arms, torso, and legs. By processing these regions separately, it can indicate which parts of the body are moving while also providing an overall movement state for each tracked person. The default configuration detects up to two people per frame and maintains separate measurement histories for their temporary tracking IDs.

The application provides two operating modes:

Visual Motion Only: Uses the webcam to detect movement, display body-region states, and record movement measurements.

Visual Motion + EMG Hardware: Adds serial reception of two EMG channels and joystick/button readings. These signals are displayed alongside the visual movement information and can be recorded for later comparison.



The project's main objectives are to:

1. Provide body-region movement classification from webcam input.
2. Maintain separate movement measurements for multiple detected people.
3. Display movement patterns and optional external sensor signals.
4. Collect manually labeled CSV data for subsequent review and analysis.

Trial labels identify the activity being performed during recording. They are supplied by the operator and do not control the movement detector. Similarly, EMG readings are displayed and recorded independently; the visual movement classification is calculated from camera landmarks.

2. Requirements

Software Requirements

Requirement	Basis in the code
Python 3.10 or later	Minimum version for some important functions
OpenCV (cv2), MediaPipe (mediapipe), NumPy (numpy)	Required imports for camera capture, graphical windows, pose inference, and array processing. Package versions are not pinned.
pyserial for hardware mode	Imported inside EmgReceiver; visual-only mode does not require it.
Desktop graphical environment and terminal input	Startup uses clickable OpenCV windows; hardware mode prompts for port and baud rate in the terminal.
models/pose_landmarker_full.task beside main.py	Required local MediaPipe model asset; startup raises FileNotFoundError if missing. The script does not download it.
Write access beside main.py	Recording creates a data directory and CSV files there.

No operating system is explicitly enforced. Windows-style COM ports appear in prompts and serial-port prioritization, but the code does not require a tested/specific operating-system support.

Role of Software Components

OpenCV: Opens the webcam, reads frames, converts image colors, draws the skeleton and graphs, displays graphical windows, and receives mouse and keyboard input.

MediaPipe: Runs the Pose Landmarker model and returns the body landmarks used for person matching and movement measurement.

NumPy: Creates and combines the image arrays used for the camera display and graph panels.

PySerial: Finds available serial ports and reads EMG/joystick data in hardware mode. It is imported only when the serial receiver starts.

The script also uses Python's standard library for background threading, communication between threads, CSV writing, timestamps, file paths, and measurement histories. These modules do not require separate installation.

Required Project Files and Storage

main.py contains the application logic. The separate model file, models/pose_landmarker_full.task, supplies the pose-estimation model used by MediaPipe.

The model path is resolved relative to the location of main.py, rather than the terminal's current directory. The model must therefore be placed inside a models folder beside the script. The program checks whether this file exists and reports an error if it is missing.

The data folder is created when recording begins. It stores movement CSV files and, in hardware mode, EMG CSV files. Write access to the project directory is therefore required for recording.

Setup and Run

Arrange the project as follows; data/ is created automatically when recording starts:

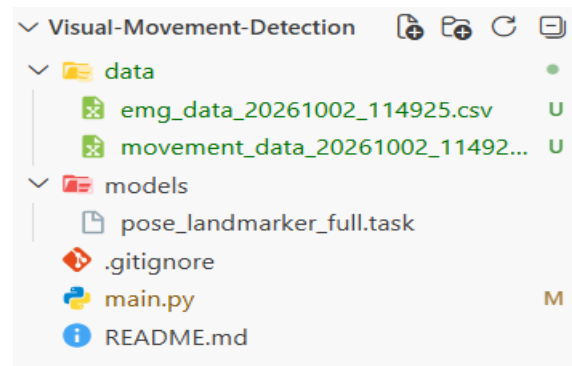
project/

├── main.py

├── models/

│ ├── pose_landmarker_full.task

└── data/



Install packages corresponding to the imports, then run from a terminal in the project directory:

```
python -m pip install opencv-python mediapipe numpy
```

Only for Visual Motion + EMG Hardware:

```
python -m pip install pyserial
```

```
python main.py
```

```
1   mediapipe==1.0.0
2   numpy==2.x.x
3   opencv-python==5.x.x
4   pyserial==3.5
```

These commands reflect the code's dependencies, rather than a supplied, version-tested environment. The required model must be provided separately.

1. Click Visual Motion Only or Visual Motion + EMG Hardware in the startup window. ESC cancels startup.
2. In hardware mode, enter a serial port (for example, COM4) or press Enter to scan available ports. Enter the device's baud rate or accept 115200. Invalid integer input falls back to that default.

```
Serial port (for example COM4; press Enter to auto-detect): COM4
Baud rate [115200]: 115200
```

3. The webcam uses `CAMERA_INDEX = 0`; edit that constant if a different camera index is required. A camera-open failure raises an error; a frame-read failure stops capture.
4. Use the keys listed in Section 3 to label trials and start recording. Paths and selected labels are printed in the terminal.

The startup prompt supplies the actual serial port

Hardware Requirements

Hardware	Purpose
Computer running the required Python libraries	Executes camera capture, pose inference, motion processing, and visualization.
Webcam accessible to OpenCV	Supplies the image frames used for movement detection.
Display, mouse, and keyboard	Allow mode selection, viewing of outputs, trial labeling, and recording control.
Compatible serial-connected device, for hardware mode	Sends two EMG sample streams and joystick/button values in the format expected by the parser.

First-Run Checklist

Before starting a session:

1. Confirm that the required libraries are installed in the Python environment used to run main.py.
2. Confirm that the model file is in the expected models directory.
3. Connect the webcam and check the configured CAMERA_INDEX, which defaults to 0.
4. Run the script from a terminal so that hardware-mode prompts and recording messages remain accessible.
5. Select the appropriate capture mode.
6. For hardware mode, enter the serial port and the baud rate expected by the device. Pressing Enter accepts automatic port scanning and the default baud rate, respectively.
7. Confirm that the camera display appears. In hardware mode, also check whether the status reports valid data reception before relying on the sensor plots.

3. How It Works

Inputs and Outputs

The webcam supplies a sequence of images. MediaPipe converts each image into pose landmarks containing image coordinates and visibility information. The movement detector uses their two-dimensional positions to calculate displacement.

In hardware mode, the application also receives serial rows containing a sequence number, device timestamp, ten samples from each EMG channel, joystick X/Y values, and a button value.

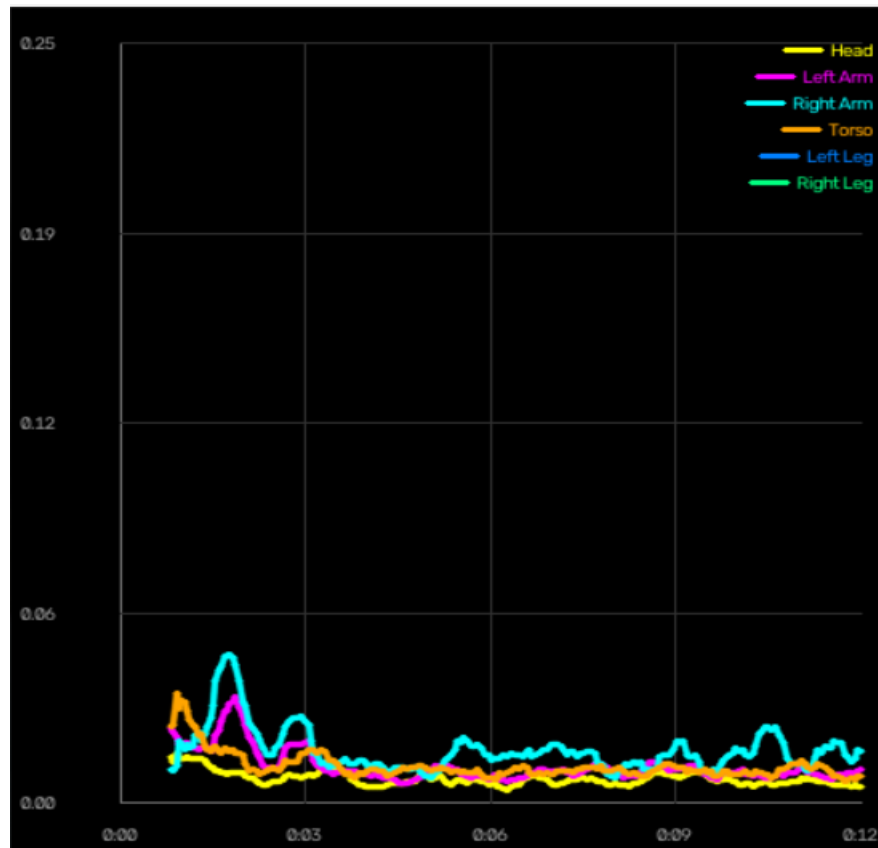
The application produces three main forms of output:

- A camera display showing body-region movement and overall movement summaries.

- Graphs showing the active person's movement history and, when enabled, EMG and joystick signals.

- CSV files containing measurements and labels collected during recording.





Important Components in main.py

Although the project is implemented in one script, its responsibilities are separated into functions and classes.

Component	Responsibility
<code>choose_capture_mode()</code>	Displays the startup choices and collects serial settings for hardware mode.
<code>PersonTrack</code>	Stores one person's region trackers, trial label, smoothing state, and plot history.
<code>PersonTrackManager</code>	Associates detected poses with temporary tracking IDs across frames.
<code>RegionTracker</code>	Calculates one region's motion and maintains its movement state.

Geometry and smoothing helpers	Check landmark reliability, calculate body scale, adjust coordinate proportions, and smooth measurements.
EmgReceiver	Receives serial rows on a background thread and queues valid data.
EmgState	Maintains recent sensor samples and the EMG RMS histories.
MovementCsvLogger and EmgCsvLogger	Write movement and external-sensor data to separate files.
Drawing functions	Build the skeleton, summaries, status indicators, and graphs.
main()	Coordinates capture, processing, display, keyboard controls, recording, and cleanup.

Camera and movement pipeline

1. **Capture and pose detection:** main() reads webcam frames, converts BGR to RGB, and calls MediaPipe Pose Landmarker in VIDEO mode with elapsed millisecond timestamps. Detection, presence, and tracking confidence settings are all 0.5. Inference uses the original image; mirroring is applied afterward for display, preserving anatomical left/right labels.

2. **Person matching:** PersonTrackManager greedily matches the closest detection/track centroids within $3 \times$ body scale. Unmatched detections receive new IDs. Missing tracks retain their last states for up to 10 missed frames and are removed when the count exceeds 10. Matching uses raw pose coordinates; aspect correction occurs later in motion processing. IDs are temporary and can change after absence, occlusion, or close crossings.

3. **Position preparation:** smooth_landmarks() applies exponential smoothing with weight 0.5, or 0.3 for knees and ankles. aspect_correct_landmarks() adjusts coordinates used for measurement, so image width and height have a consistent distance scale. Drawing retains normal image coordinates. Body scale prefers shoulder width, then hip width and left/right shoulder-to-hip distance, with a minimum of 0.05. smooth_body_scale() smooths that reference and retains the previous value when no new scale is available.

4. **Visibility checks:** A landmark must have visibility at least 0.50 and coordinates inside [0, 1]. A region is RELIABLE when all its landmarks qualify, PARTIAL when a connected segment remains, and UNRELIABLE otherwise. Insufficient visibility or a missing motion baseline produces UNCERTAIN, rather than a valid zero-motion measurement.

5. Motion measurement: Each RegionTracker handles head, left/right eye, left/right arm, torso, or left/right leg. Non-head regions use the maximum displacement among landmarks shared with the previous measurement. Head motion combines average landmark displacement with nose movement relative to the ear midpoint and change in the ear-to-ear vector, when available. Raw displacement is divided by body scale, then averaged over up to 8 measurements. The resulting motion values are dimensionless displacement ratios, not physical speed.

6. State decision: A region becomes MOVING after 3 consecutive measurements at or above its movement threshold. Once moving, it requires 8 consecutive measurements at or below its stationary threshold to return to STATIONARY. Between thresholds, the moving state persists. After the initial uncertain baseline, valid measurements default to stationary until movement persistence is satisfied. Overall state is moving if any measurable region moves, stationary if measurable regions exist and none moves, and uncertain if no region has a valid measurement.

Understanding Motion Calculation

A landmark's displacement is the distance between its current position and its previous measured position. For most regions, the largest displacement among the available shared landmarks drives the reading.

For example, the arm region includes the shoulder, elbow, and wrist. If the wrist moves while the shoulder remains nearly fixed, using the maximum displacement allows the wrist movement to contribute directly instead of being diluted by averaging it with the less active landmarks.

The displacement is divided by a body-size reference. This reference is normally shoulder width, with hip width and shoulder-to-hip distances used as alternatives. Normalization helps account for changes in apparent body size when a person is nearer to or farther from the webcam.

The resulting motion value is averaged over recent measurements before classification. It represents displacement relative to body size; it is not a measurement in centimeters or a physical velocity.

Head movement uses additional geometry. Besides landmark displacement, the calculation considers the nose relative to the midpoint of the ears and changes in the ear-to-ear vector. These measurements are intended to represent changes associated with head turning.

Visibility and Movement State

Visibility and movement state describe different aspects of a measurement.

Observability indicates whether sufficient landmarks are available: reliable, partial, or unreliable.

Movement state indicates the detector's decision: moving, stationary, or uncertain.

A partially visible region can still be measured when a meaningful connected segment is available. When insufficient information remains, the region's motion baseline is reset. The next usable observation establishes a new baseline before displacement can be calculated.

The separate movement and stationary thresholds provide hysteresis. A region must repeatedly exceed its movement threshold before entering the moving state. Once moving, it must repeatedly fall below its stationary threshold before returning to stationary. This persistence prevents a single threshold crossing from immediately changing the state.

Current thresholds applied to smoothed, normalized motion:

Region	Moving threshold	Stationary threshold
Head	0.0274	0.0261
Left eye	0.0745	0.0710
Right eye	0.0777	0.0740
Left arm	0.0504	0.0480
Right arm	0.0451	0.0430
Torso	0.0336	0.0320
Left leg	0.0564	0.0537
Right leg	0.0482	0.0459

EMG and joystick pipeline

EmgReceiver reads serial data on a background thread and queues valid rows without blocking the camera loop. Auto-detection prioritizes available COM3/COM4 ports, then tries other listed ports until one opens; opening a port does not prove valid data is arriving. The receiver retries when ports are unavailable or after a serial disconnection.

parse_emg_line() expects these 25 comma-separated integer fields:

```

seq,

timestamp_ms,

emg1_0,...,emg1_9,

emg2_0,...,emg2_9,

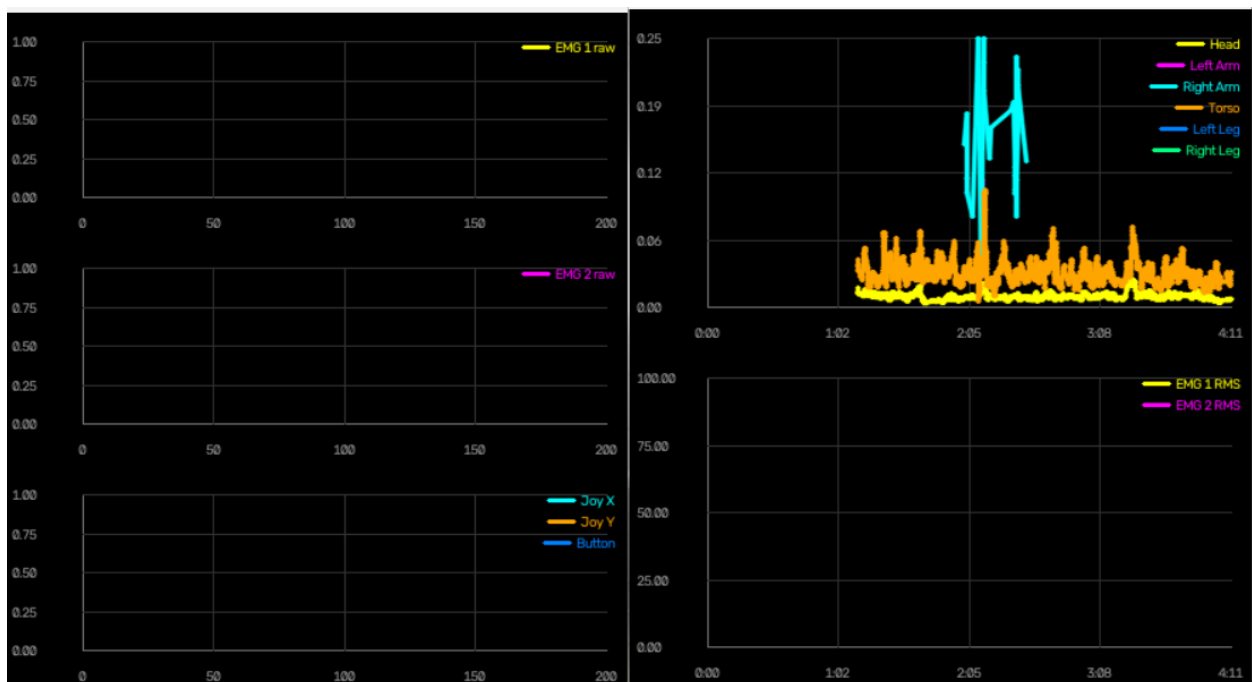
joy_x,

joy_y,

btn

```

Blank, short, or non-integer rows are discarded; extra trailing fields are ignored. Each accepted row carries ten samples per EMG channel and one joystick/button update. The receiver adds a PC receipt timestamp using the same elapsed-time origin as the camera.



EmgState stores the latest 200 samples per EMG channel and 200 joystick rows. Every 50 EMG samples, compute_rms() calculates the square root of the mean squared deviation from that batch's mean. This removes a constant batch offset and creates an RMS trend in raw sensor units. No conversion to volts, device sampling-rate configuration, or hardware synchronization is implemented.

Comparing Movement and EMG Data

Movement frames and serial rows arrive independently. The background receiver allows serial data to be collected while the camera loop continues processing images.

Both pipelines use elapsed PC time measured from the same session origin. The serial timestamp records when a row is received, while the movement timestamp corresponds to camera processing. These timestamps support later alignment, but the two files should not be compared simply by matching row numbers.

The EMG RMS plots summarize variation within batches of samples after subtracting each batch's mean. The raw plots remain useful for inspecting the received waveform. The EMG CSV stores the received raw samples; the displayed RMS values are not saved as separate CSV columns.

Display, controls, and outputs

The main window shows a mirrored camera image with a colored skeleton, overall movement summaries, date/time, and the active person's movement plot. Red indicates moving, green stationary, orange uncertain, and purple marks partial stationary skeleton connections. Eye regions are tracked and logged but omitted from the movement plot. Its vertical range is 0–0.25, with larger values visually clamped.

Hardware mode adds an EMG RMS plot and a separate Live COM EMG + Joystick window with raw channel and joystick/button plots. Connection status distinguishes no connection, waiting for valid data, receiving data, and data stopped for more than two seconds. Movement and RMS plots use time since camera-session startup, including periods when CSV recording is off.

Key	Action
R / r	Start/stop CSV recording when CALIBRATION_MODE = True.
Tab	Cycle the active person for trial labeling and movement plotting; the terminal prints the active ID.
0, 1, 2, 3, 4	Assign NO TRIAL, STILL, RIGHT_ARM, LEFT_ARM, or HEAD.
5, 6, 7, 8, 9	Assign RIGHT_LEG, LEFT_LEG, WHOLE_BODY, SITTING_STILL, or LYING_STILL.
S / s	Assign LYING_SIDEWAYS_STILL.
q / Esc	Exit capture and release camera, windows, receiver, and open log files.

Calibration controls:

R = Start/stop CSV recording

Tab = Switch which tracked person is "active" (the one 0-9/s label)

0 = NO TRIAL

1 = STILL

2 = RIGHT_ARM

3 = LEFT_ARM

4 = HEAD

5 = RIGHT_LEG

6 = LEFT_LEG

7 = WHOLE_BODY

8 = SITTING_STILL

9 = LYING_STILL

S = LYING_SIDEWAYS_STILL

Q or Esc = Exit

Trial labels are manual annotations and never change detection. Each new track begins with NO TRIAL, reselect the label if a person receives a new ID.

Operating a Recording Session

After the camera starts, select the person to label using Tab. The selected person determines which movement history is shown, and which trial label is changed by the labeling keys.

Choose the trial label that describes the activity being performed, then press 'R' to start recording. All detected people are recorded, each with their own tracking ID and current label; recording is not restricted to the active person.

Labels can be changed during recording as the activity changes. Pressing 'R' again closes the current files. Starting another recording interval creates new files while the camera session continues.

Temporary IDs should be checked when a person leaves the frame, becomes occluded, or crosses closely with another person. If a new track is created, its label starts as NO TRIAL and must be selected again.

Recorded Files

Recording creates the following files beside the script:

data/movement_data_YYYYMMDD_HHMMSS.csv: One row per detected person per recorded frame, containing elapsed timestamp_ms, frame number, track ID, trial, body scale, head raw/normalized motion, every region's smoothed motion, observability and state, and overall state. Unavailable motion fields are blank. No rows are written for missing tracks during their missing period.

data/emg_data_YYYYMMDD_HHMMSS.csv: Hardware mode only, one row per received device row, containing PC elapsed timestamp_ms, device timestamp, sequence number, twenty raw EMG samples, joystick X/Y, and button value. RMS values are displayed but not written to this CSV.

Both files use the same recording-start filename timestamp and PC elapsed-time convention, their different row rates are preserved. Alignment for later comparison use PC timestamp_ms, rather than treating rows as paired samples. Existing filenames receive numeric suffixes. Each new recording interval creates new files, but elapsed timestamps continue from session startup. The script saves CSV data only; it does not save video or plot images.